

Kubernetes Resource Capacity Planning on Cloud

■ [TechCohere, January 20, 2025](#)

Agenda

- 1) Current Challenges
- 2) Quick Checklist
- 3) Identifying Application Tier and Geography
- 4) High Availability and Disaster Recovery Planning
- 5) Choosing Right Instance Types
- 6) Requests and Limits -Strategy
- 7) Performance Testing Strategy
- 8) Case Study

Introduction

In the fast-moving containerized world, every system including Kubernetes has some architectural limits. This can either cause you performance issues on your cluster because of low resources or ending up huge spendings for high resource utilization.

Right capacity planning is not just going to benefit your cost or optimize your resources, but it is also a way of saving the environment.

This Session is going to cover some of the best practices to consider while planning capacity for your resources.

Current Challenges

1. Not using the right nodes(instance types) of the right use cases. Over provisioning or underusing Cpu/Memory.
2. Challenges with number of pods running on nodes.
3. Challenges with resource constraints in pods.
4. Node Auto provisioning without proper capacity planning.
5. Resource Governance
6. Not using tags and labels properly.
7. SLA Breaches and Developer Challenges
8. Effective use of Autoscaling parameters.

Quick Checklist

Before starting with capacity planning, we need to do lot of ground works and collect enough details on the application.

If you are looking at a migration project where your existing workloads are running in other Kubernetes platforms like OpenShift or your workloads are hosted as VMs in on premises, then please do the following.

Collect the volumetric data of the application. This includes the Number of requests/loads on production(Min/Average/Max) and Latency details. Collect the expected and predicted load in the future say for next 6 months or 1 year. This will also help you setting up scaling parameters.

Understand the current landscape of the application. How many front-end application/services, backend services, databases, dependent systems and third-party products and services.

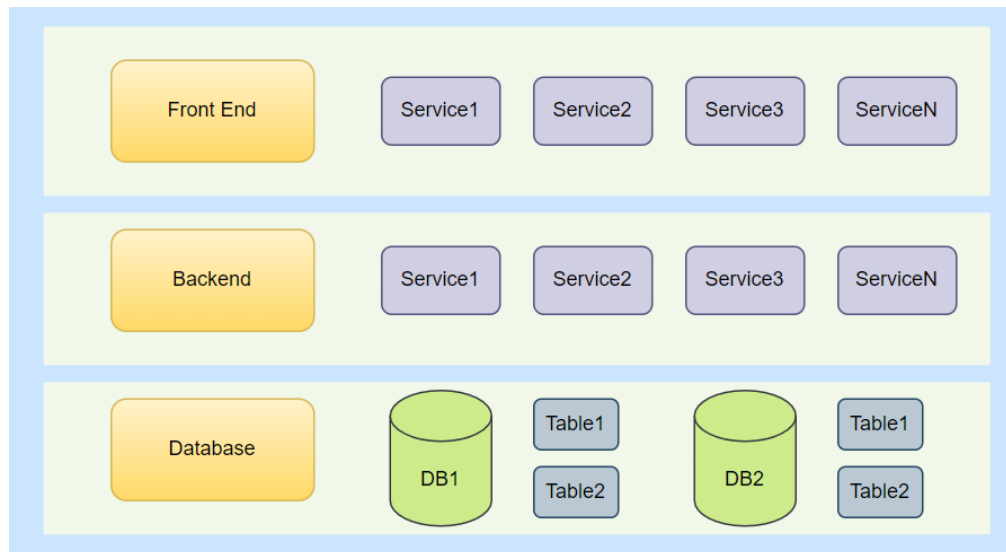
Capture current infrastructure details : How many instances/nodes/servers are running for production and how many services/applications are hosted along with the CPU/Memory/Storage/Network details. When we are talking about the CPU/Memory/Storage/Network we need to understand what the allocated resources is, what is the average utilization and maximum utilization. Most of the scenarios, it is little challenging to collect the details at the individual services level but that is fine.

The above details will give you an overall holistic overview of your current landscape of your system.

Resource Classification

If you are looking at a project from a scratch or modernizing a mainframes application above, we might slightly need to change our way of approach as we don't have any past data for prediction.

Based on the information, we need to strategies the planning. See the below diagram, the number of resources for each layer helps you to plan and define how many clusters you need and how many nodes you need to create. We will talk more about this in the later part of this document.

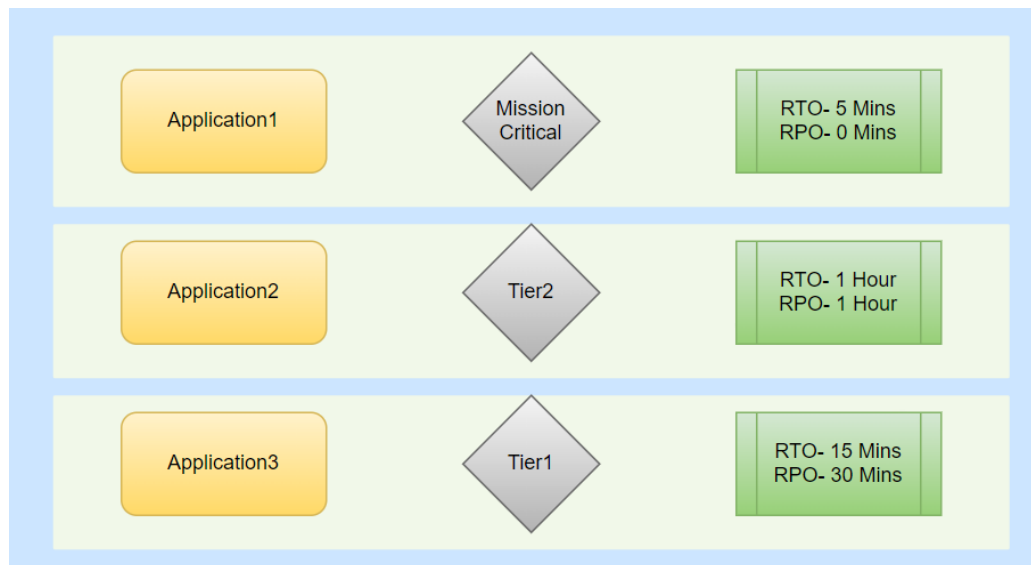


Identify Your Application Tier

This is your first step for capacity planning. Identifying and categorizing your application tier is mainly used for your SLA Requirements. Some of your services might be mission critical and need high availability requirement of 99.99 and some might not be much critical where the availability might be little low.

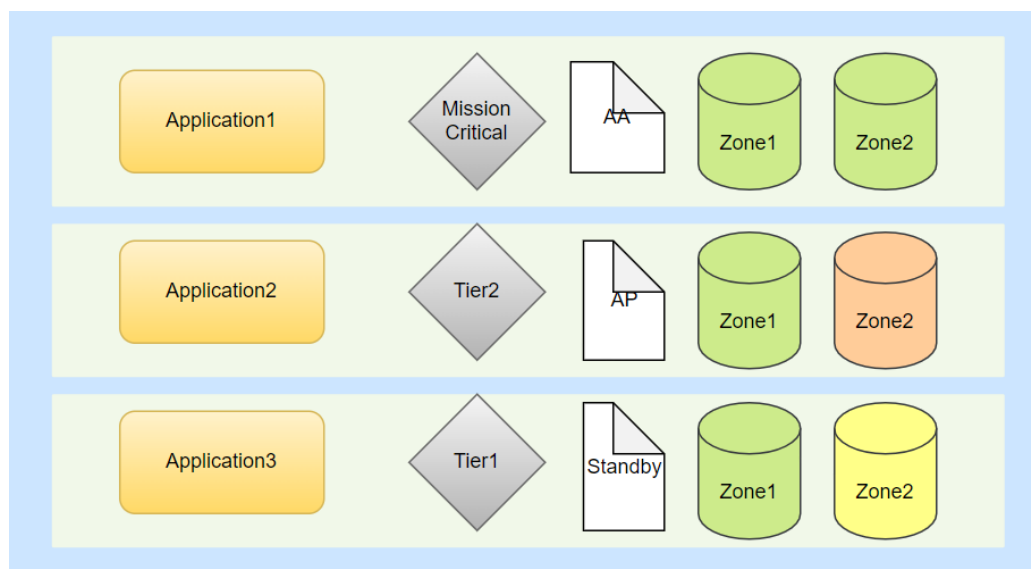
This categorization helps you plan your capacity, scaling and HA-DR requirements and ensures your SLA is not breached.

Capacity Planning not only going to help us reduce costs, but it also brings happy customers too.



Identify your Geography:

Once you have identified your application tier, we need to identify the region where your applications/services are going to be hosted. This will help you choose whether you are looking at a Global Solution with multiple locations or want your applications to run on multiple zones inside a single region or want to run on single zonal cluster. This decision will be also used in defining your DR Strategy.



AA- Active Active Setup, AP- Active Passive Setup, Standby- Warm Standby

This will let you decide how many minimum pods you need to plan for your services and application. As per above diagram if you are looking for a high availability of your application, we

need to ensure that the application has 2 pods and distributed across 2 zones. If both the pods run in same zone, this is not going to bring high availability.

Choosing Right Instance Types:

Every cloud provider share variety of instance types and customer can choose the best fit for the use case. Some applications like gaming needs more memory and graphical processors and some applications are high cpu extensive. If you don't have any specific requirements, please go for a general compute instance which has the good ratio of cpu and memory.

Your instance type will be judged/selected based on the following parameters:-

1. Type of environment. For dev and lower environments go for a smaller instance types.
2. HA and DR – Cost of instances may vary from region to region. Please plan accordingly.
3. NFRs like latency, throughput, real time events.
4. Type of Services. For example, Databases - Storage Optimizes instances, Cache Services – Memory Optimized instances etc.

Approach to Request and Limits:

So far, we have identified the number of applications/services we need to consider for the capacity planning, tier of the application, the region where we are going to deploy our application and the options available for the instance types.

Before moving into next topic, you need to understand the couple of terms used in Kubernetes.

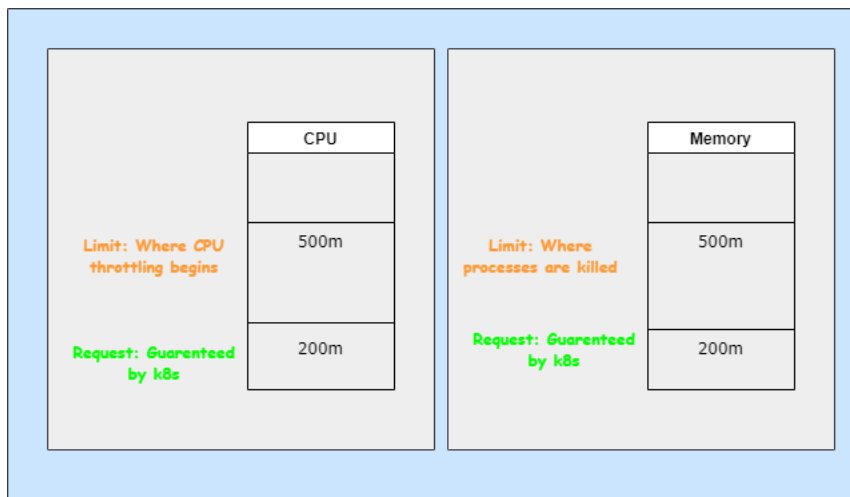
Request: Minimum guaranteed amount of a resource(cpu/memory) that is reserved for a container

Limits: Maximum amount of a resource to be used by a container.

ResourceQuota: Policy to set constraints on the total resource usage per namespace.

LimitRange: Policy to constrain resource allocations (to Pods or Containers) in a namespace.

These 4 key properties let you control your applications from overconsuming the resources in the Kubernetes cluster.



Requests and Limits for both Memory and Cpu, will not work in the same way inside the containers. But both works differently if there is a shortage or overallocated. Please check the below example.

Resources	Low	High
Request CPU	Starvation, Might not get cpu cycle	Inefficiency, Other pods will not be scheduled as the resources are on hold by single resource
Request Memory	High Risk to be killed, Resource will be pulled by other pods	Inefficiency, Other pods will not be scheduled as the resources are on hold by single resource
Limit CPU	CPU Throttling	Inefficiency, other pods starve
Limit Memory	Pod will be killed by kernel	Inefficiency, other pods starve

So, we must be very careful in choosing the right requests and limits. Let's see a potential approach to handle the Requests and Limits

We were talking about the application tier classification a bit before. We need to

Tier	Request	Limit
Critical / Highly Available	99.99th percentile + 100% headroom	2x request or as higher if resources available
Production / Non-critical	99th + 50% headroom	2x request
Dev / Experimental	95th or consider namespace quotas*	1.5x request or consider namespace quotas*

This best practice of analyzing historical usage patterns typically provides both a good representation of the future and is easy to understand/introspect. Applying extra headroom allows for fluctuations that may have been missed by your historical sampling. We recommend measuring usage over 1 week at a minimum and setting thresholds based on the specific availability requirements of your pod.

If you are not clear, Lets see this with an example.

Service	Desciption	Tier	SLA	Requests Per Minute	Request CPU	Limit CPU
Service A	Payments	Mission Critical	99.99	1000	1vCPU	2vCPU
Service B	Accounts	Non Critical	99.5	200	0.2 vCPU	0.5vCPU

There are 2 services one is critical, and another one is noncritical.

Scenario1- For the Payments Service, when your vCPU is 0.5, you can achieve your 99.99 Percent, however we are going with 2x as the application is critical and adding headroom for spikes. Trade off here is cost, but still the application is critical, and we must plan for it.

Scenario2- For the Accounts Service, the SLA is low, and your Request and Limits CPU are planned with your SLA with additional headroom.

The next question comes to your mind, how is this even possible. How can I trust when my vCPU is 0.5, I was able to achieve my SLA. This is where the performance testing comes into the picture. Performance testing or load testing is one of the key factors for the Capacity Planning. There cannot be a proper Capacity Planning without the Performance testing.

Before moving on to performance testing, lets quickly check how ResourceQuota and LimitRanges work.

Resource Quota and LimitRanges:

Both Resource Quota and LimitRanges help you set your resource constraints at Namespace level. In the fast-moving multitenancy world it is very vital to use these constraints in a better way and efficient way.

Resource Quota is huge, we have lot of constraints available. But here mostly we are going to discuss on the request and limits on cpu and memory. You can also set storage limits, limits based on number of pods, services and other Kubernetes resources.

Resource Name	Description
limits.cpu	Across all pods in a non-terminal state, the sum of CPU limits cannot exceed this value.
limits.memory	Across all pods in a non-terminal state, the sum of memory limits cannot exceed this value.
requests.cpu	Across all pods in a non-terminal state, the sum of CPU requests cannot exceed this value.
requests.memory	Across all pods in a non-terminal state, the sum of memory requests cannot exceed this value.

Consider a small example below. There are 2 pods running in a single namespace.

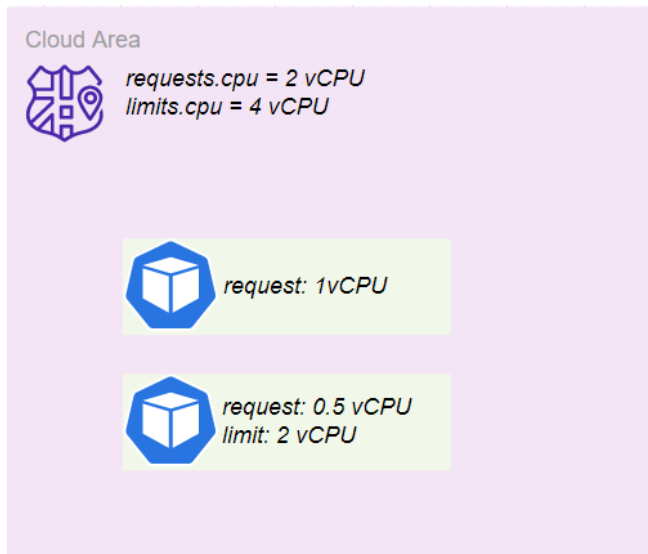
Scenario1:

For the Pod1, there are no limits setup. In this scenario, the request for the pod is set at 1vCPU and when the load increases, maximum it can pull resources upto 3.5 vCPU as the limit set at the namespace level is 4vCPU. 0.5vCPU is already allocated for another pod.

Scenario2:

For the Pod2, the requests and limit are set at pod level and cpu throttling happens when the limit reaches 2vCPU.

Note: You can also limit number of pods at namespace level.

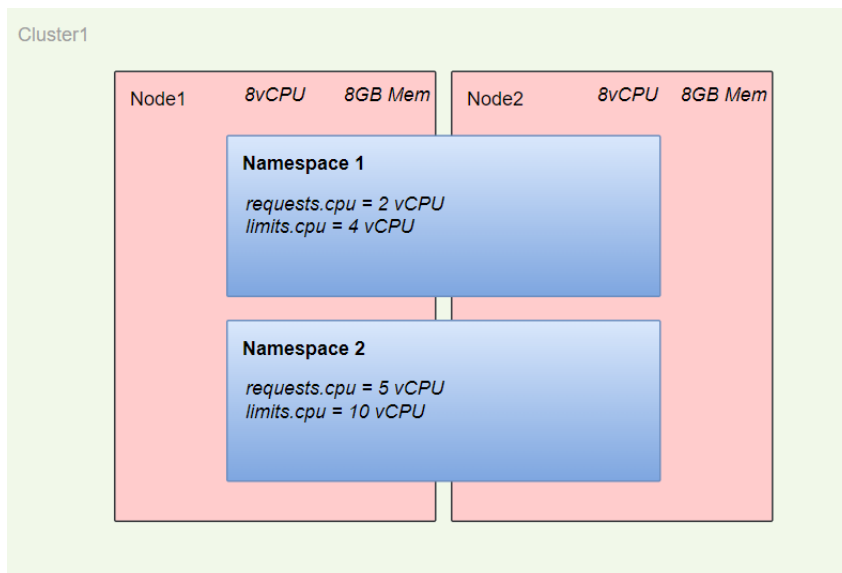


Now it's the little bit of tricky part.

Cluster Autoscaling and ResourceQuota:

Resource Quota has nothing to do with your node instances. See the below example. For the namespace 2, the limits.cpu are higher than the node vCPU size which is 8vCPU. If Cluster Autoscaling is enabled and if the pods were unable to schedule in the namespace 2, then the kube-scheduler will look for the node where the capacity is available and if all the nodes are full then the cluster autoscaler will create a new node within the cluster.

AWS, and GKE has node groups where you can govern the number of nodes and can add node level constraints. Some clients also prefer **Karpenter** for running on baremetal solution and OpenShift for efficient node management.



LimitRanges

Limit Ranges in Kubernetes allow administrators to define default resource requests and limits for Pods, Containers, and PersistentVolumeClaims (PVCs) within a namespace. This ensures that individual Pods or containers do not exceed specified resource limits, preventing any single Pod from consuming excessive resources. By using Limit Ranges, you can control resource usage for Pods, containers, and PVCs, ensuring efficient and fair resource distribution across your cluster.

Why LimitRanges if we have Resource and Limits at Pod/Container level?

The scope of pod is limited. You benchmark a pod and assign resource constraints based on the capacity. But when it comes to governance, it is little challenge to check each pod and understand the constraints and if there are no OPA and Gatekeeper rules, there are high chances that the Dev/Track team to miss the resource constraints on the pod and container level.

As a admin or as an architect, we can enforce this at the namespace level. If we set LimitRanges at namespace level, this will be applicable for all the pods within the namespace.

Cpu and Memory for LimitRanges can be set at different levels including Containers, Pods and Storage(PVC) and maxLimitRequestRatio defines the max limit can be consumed at namespace level.

Which will be applicable if we are setting Resource Constraints at namespace level(using LimitRanges) and at Pod level(request, limits at pod level)?

LimitRanges is like an admission controller and executes resource constraints like a default value. This default value can be overridden by the resource constraints set on Pod and Container level but the maxLimitRequestRatio at the namespace level will still not allow more than the value set.

Performance Testing Kubernetes Resources:

Performance testing will help to achieve the below main things with respect to the capacity planning exercise.

1. Measuring Key Performance Indicators like Latency, Throughput, CPU, Memory, Network, Saturation and Concurrent Users.
2. Autoscaling Parameters
3. Help you decide/Judge the number of pods, with its request and limits.

Key Performance Objectives are

- *Speed – Response time, Throughput (Transaction Volume – No of Transactions per sec/min/hour), concurrent users*
- *Reliability (or Availability) – availability of the system for longer durations under sustained loads (i.e. through the business hours) without any performance impacts*
- *Scalability – ability of the Application and Infrastructure to scale to higher user loads and transaction volumetrics (beyond the regular loads,).*

Key Performance Test designs to address above objectives are

- *Avg and Peak Load Tests*
- *Endurance Tests*
- *Max Load Tests (currently we are testing 1.5 times the current expected)*
- *Test early in the life cycle i.e. conducting shift left tests in lower environment like SIT/SAT and have some baseline and raise defects early in the testing phases*

Standard Metrics that are monitored across any Performance Test using the availability tool set: JMeter/LoadRunner, Dynatrace, CloudWatch, ELK.

Capacity Planning tests can be achieved by validating the Max Load Tests – (We may need to have a dedicated environment for this for flexibility). In some application, the stress test can go up to 3X – 4X as required.

Results (vCPU, Mem)					Remarks
1.5, 2.5	1, 2	0.75,1.5	0.5,1	0.25, .512	
Latency @95th Percentile in Milliseconds					
1989	2502	6562	22630	39981	NA
1692	7897	6380	13336	24817	NA
4564	15970	11469	26951	51159	NA
1248	6778	2638	5475	10307	NA
1722	9607	2604	6489	10136	NA
65	192	284	872	2326	NA
UAT1	UAT1	UAT1	UAT1	UAT1	NA

Client Case Study

Client Profile

- American insurance and financial services company headquartered in Pennsylvania. This Financial Organization provides high-quality investments, personalized financial advice, retirement tools and relevant market insights.

Tech Stack

- Java Microservices, Angular, GraphQL, ELK, AWS, EKS, Postgres Aurora.

Project Challenges

- No Uniformity on Pods.
- Resource Constraints are not properly setup.
- No proper Labels or Observability enabled.
- Resource governance was not established properly.
- Autoscaling strategy was not properly used.
- Overutilization of resources and cost.

Impact/Benefit

- Industry Standard Best Practices for Capacity Planning
- Multi-tenancy where applicable.
- Helped team setting up Performance KPIs and Autoscaling parameters.
- Effective use of Pod Affinities and Node Affinities.
- Separate Strategy for Non-Prod Environments and Prod Environments.
- Metrics Best Practices and Observability setup.

Impact and Benefits:

- Able to Reduce 10-15% of AWS Total Cost.
- Improved usage of Resources
- Observability Best Practices to make troubleshooting easier.
- Improved KPI Performances.

Other Key Considerations:

AutoScaling Parameters: Most of the current HPA implementations are based on CPU Utilization. It may be an ideal solution but there are at times, there can be an immediate spike in CPU will create rolling up a pod and handling these kinds of scenarios is bit challenging. Other options are creating HPA based on custom metrics(number of requests) and use AI tools for predictions(Cast AI, Harness etc.).

In GCP there is an Autopilot mode, where it uses AI capabilities to scale up and scale down the resources of its own.

Custom Policies to Manage Kubernetes Resources:

Consider using OPA or Kynervo for managing Kubernetes resources.

Appendix

Choosing Right Machine Types - <https://cloud.google.com/compute/docs/machine-resource>

Requests and Limits in Kubernetes: <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>